

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1204	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	M. HARSHA	Reg.No.:	192525108
Department:	B.TECH AI&ML	Date:	22.08.2026

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
2. Cloud Server Memory Bottleneck
A cloud service provider experiences high latency in database

queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I **M.HARSHA(192525108)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible	Logical and feasible solution	Basic solution with	Unclear or impractical solution

		solution aligned with industry standards	with minor gaps	limited feasibility	
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Question 1: Smartphone Performance Optimization

Problem: A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

1. Understanding of the Industry Problem

When a user switches apps, the SoC must reload the target app's working set (code, heap, UI state) from a cold or evicted cache state. If the app's data has been evicted from L1/L2/L3 by the previously running app or background services, every subsequent access becomes a cache miss that must be serviced from DRAM, and in the worst case from flash-backed swap. This manifests to the user as visible lag, dropped frames in the UI compositor, and delayed input response.

The core industry constraint is that mobile SoCs are power- and area-limited, so cache sizes cannot simply be scaled up without a battery and thermal cost. The redesign must therefore improve effective hit rate and DRAM efficiency rather than just add capacity.

2. Application of Engineering Concepts

L1 cache (typically 32–64 KB, split instruction/data, per-core) offers 1–2 cycle access latency and services the immediate instruction/data stream of the active app's foreground thread.

L2 cache (256 KB – 1 MB per core or per-cluster) has ~8–12 cycle latency and holds the broader working set (UI framework code, recently used objects), reducing traffic to L3.

L3 cache (2–8 MB, shared across cores) has ~30–40 cycle latency and acts as a victim cache for the whole SoC, critically holding data shared between the app process and system services (window manager, graphics compositor).

DRAM (LPDDR5/5X, several GB) has 100–300 cycle latency and orders-of-magnitude higher latency than L3, so every L3 miss during an app switch is expensive; DRAM bandwidth (tens of GB/s) also becomes the bottleneck when many evicted pages must be re-fetched simultaneously.

Level	Typical Size	Latency	Role in App-Switch Scenario
L1 (I/D)	32–64 KB	1–2 cycles	Active thread's hot instructions/data
L2	256 KB–1 MB	8–12 cycles	App working set, UI objects

Level	Typical Size	Latency	Role in App-Switch Scenario
L3 (shared)	2–8 MB	30–40 cycles	Cross-process data, foreground app priority
DRAM (LPDDR5)	6–16 GB	100–300 cycles	Full app image, background app pages

3. Solution Design

The redesign targets three levers: (a) protect the foreground app's footprint from eviction, (b) shorten the DRAM refill path, and (c) hide latency behind prefetching.

- Foreground-aware L3 partitioning: use way-partitioning or Cache Allocation Technology (CAT)-style QoS so the active foreground app is guaranteed a reserved portion of L3, preventing background services from evicting its working set.
- Predictive prefetch on app-switch gesture: the OS scheduler signals the memory controller as soon as a task-switch/recents-list tap is detected, triggering early DRAM-to-L3 prefetch of the target app's last-known working set before the UI frame is drawn.
- Victim/exclusive L3 with higher associativity and RRIP (Re-Reference Interval Prediction) replacement to retain recently-switched-from apps briefly, reducing the cost of switching back and forth (common in multitasking).
- Faster DRAM tier: adopt LPDDR5X with higher bandwidth (up to ~8.5 Gbps/pin) and enable memory bandwidth QoS so the foreground app's refill requests are prioritized over background sync/telemetry traffic.
- Zram/compressed-RAM tier between DRAM and flash swap so recently backgrounded apps stay resident in compressed DRAM instead of falling all the way to slow flash-backed paging, cutting resume latency significantly.

4. Use of Tools and Technologies

- ARM Streamline / DS-5 or Android Studio CPU & Memory Profiler to capture cache-miss and page-fault events during live app-switch traces.
- gem5 or DRAMSim2 architectural simulation to model proposed cache partitioning and quantify hit-rate improvement before silicon changes.
- Perfetto system tracing to correlate scheduler task-switch events with memory-controller latency spikes.
- Cachegrind/Valgrind for offline cache-behavior profiling of the app framework code.

5. Analysis / Expected Outcome

With foreground-priority L3 partitioning and predictive prefetch, the expected effect is a rise in L3 hit rate for the newly foregrounded app (commonly modeled as a 15–25% reduction in cold-miss penalty in simulation), which translates to app-switch latency dropping from the 300–500 ms range perceived as "lag" toward sub-150 ms, restoring smooth 60/120 Hz frame delivery.

Question 2: Cloud Server Memory Bottleneck

Problem: A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

1. Understanding of the Industry Problem

Database query latency spikes typically occur when the working set of hot tables/indexes exceeds available cache and physical DRAM, forcing the OS to service requests through page faults and, in severe cases, swap to disk. Each page fault stalls the query thread while the kernel walks page tables, and TLB misses add further overhead on every large-table scan.

In multi-tenant cloud servers, NUMA effects compound this: a query thread on one socket may repeatedly fault on memory pages physically attached to a remote socket, adding cross-interconnect latency on top of the fault itself.

2. Application of Engineering Concepts

Cache hierarchy (L1/L2/L3) exploits temporal and spatial locality automatically in hardware at nanosecond granularity; it is transparent to software but limited to a few MB–tens of MB of shared L3, far smaller than typical database working sets (tens to hundreds of GB).

Virtual memory with paging extends the addressable working set far beyond physical DRAM by mapping pages to disk, but a page fault costs on the order of milliseconds (disk-backed) versus nanoseconds for a cache miss — roughly five to six orders of magnitude slower — making paging the dominant source of the observed latency once the working set outgrows RAM.

TLB misses sit between these two: because the page table itself must be walked (multiple memory accesses) on every TLB miss, large working sets with small 4 KB pages generate significant TLB-miss overhead even without a full page fault.

3. Solution Design

The recommendation is a layered improvement addressing cache efficiency, paging overhead, and the DRAM/storage boundary simultaneously.

- Huge pages (2 MB / 1 GB) for the database buffer pool to cut TLB misses by up to 512× per mapped region, directly reducing page-walk latency for large sequential/index scans.
- NUMA-aware memory allocation and thread pinning (e.g., numactl) so each query worker's data resides on its local DRAM node, avoiding remote-access penalties.
- Expand effective L3 through a larger shared last-level cache or an SoC with 3D V-cache-style stacked SRAM, increasing the portion of hot indexes served without a DRAM round trip.
- Introduce a persistent-memory tier (e.g., byte-addressable non-volatile DIMMs / battery-backed NVDIMM) between DRAM and SSD storage, giving the buffer pool a much larger "warm" tier with latency far below disk-backed paging, reducing reliance on traditional swap.
- Tune the OS page-replacement and swappiness policy so database buffer pages are never preferentially swapped ahead of less-critical OS caches.

4. Use of Tools and Technologies

- Linux perf and vmstat to capture page-fault rate, TLB-miss counters, and cache-miss ratios under production query load.
- Intel VTune Profiler / AMD uProf for hardware-level cache and memory-latency analysis.
- numastat and numactl to diagnose and correct NUMA locality problems.
- Database-native tools (e.g., MySQL Performance Schema, PostgreSQL pg_stat_* views) to correlate slow queries with buffer-pool miss events.

5. Analysis / Expected Outcome

Adopting huge pages typically cuts TLB-miss-related overhead substantially for large scans, while NUMA-aware placement removes cross-socket latency penalties; combined with a persistent-memory warm tier, page-fault-driven stalls (millisecond-scale) are converted into sub-microsecond persistent-memory accesses for the majority of previously swapped pages, materially lowering p99 query latency.

Question 3: Autonomous Vehicle Data Processing

Problem: An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

1. Understanding of the Industry Problem

Autonomous driving compute (sensor fusion of LiDAR, radar, and multiple cameras) is safety-critical and hard real-time: perception and control loops must complete within a

fixed deadline (often under 10–100 ms), and any unpredictable latency spike can delay obstacle detection or braking decisions.

The system must also tolerate power interruptions (e.g., a momentary power-rail glitch) without losing safety-critical calibration or state data, and must operate reliably across automotive temperature and vibration extremes.

2. Application of Engineering Concepts

SRAM offers deterministic, very low latency (single-digit nanoseconds) and no refresh overhead, but its low density and high cost per bit restrict it to small on-chip caches and scratchpad buffers for the perception pipeline's innermost loops.

DRAM (automotive-grade LPDDR5) provides the density needed to buffer multiple simultaneous high-resolution sensor streams and intermediate neural-network activations, at the cost of higher latency and periodic refresh cycles that can introduce small, non-deterministic stalls.

MRAM is non-volatile, has no refresh requirement, offers latency closer to SRAM/DRAM than to flash, and is inherently more radiation- and temperature-tolerant — making it well suited to storing calibration parameters, safety state, and system logs that must survive power loss instantly.

Technology	Latency	Volatility	Best Use in AV Stack
SRAM	~1–2 ns	Volatile	L1/L2 cache, DSP/accelerator scratchpad
DRAM (LPDDR5)	~10–20 ns	Volatile (refresh)	Sensor frame buffers, NN activations
MRAM	~10–30 ns	Non-volatile	Calibration data, safety state, instant recovery

3. Solution Design

The proposed hierarchy places each technology where its physical properties best match a specific real-time or reliability requirement.

- Tier 0 — SRAM scratchpads (KB–low MB) inside each sensor-fusion accelerator/DSP core, guaranteeing deterministic single-cycle access for the innermost fusion and filtering kernels.

- Tier 1 — On-chip SRAM L2/L3 shared cache across compute clusters, sized to hold one full sensor-frame's worth of intermediate features to avoid stalling the perception pipeline on off-chip access.
- Tier 2 — High-bandwidth automotive DRAM (LPDDR5/HBM2E on premium platforms) as the main buffer for raw multi-sensor streams and CNN weight/activation working sets, with dual-channel or wide-bus configuration to maximize aggregate bandwidth.
- Tier 3 — MRAM for non-volatile, instant-on storage of sensor calibration matrices, redundant safety-state checkpoints, and black-box event logs, so the system can resume safe operation immediately after any power interruption without a slow reboot from flash.
- A dedicated, isolated memory channel (bandwidth partitioning) is reserved for the safety-critical control loop so that bulk data streaming (e.g., logging to storage) can never starve the real-time perception path — a QoS mechanism at the memory controller.

4. Use of Tools and Technologies

- gem5 or DRAMSim3 to simulate worst-case latency and bandwidth contention across sensor-fusion workloads.
- AUTOSAR / hardware-in-the-loop (HIL) test rigs to validate real-time deadline compliance under fault-injection scenarios (including simulated power glitches).
- Vendor MRAM/SRAM datasheet-based memory compilers (e.g., NVSim) to size on-chip scratchpad capacity against timing budgets.

5. Analysis / Expected Outcome

This hierarchy is designed to keep the innermost fusion loop within SRAM's nanosecond-latency envelope, sustain the tens-of-GB/s bandwidth needed for concurrent camera/LiDAR/radar streams via wide DRAM channels, and guarantee zero-loss recovery of safety-critical state via MRAM — directly satisfying the bandwidth-maximizing and latency-minimizing goals of the problem statement.

Question 4: AI Inference Edge Device

Problem: An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

1. Understanding of the Industry Problem

Edge AI devices (smart cameras, wearables, IoT sensors) run on tight power budgets, often battery-powered, and are expected to load and begin inference within a very short wake-up window. Traditional designs load the full model from NAND flash into DRAM

at every boot/wake cycle, which is both slow (flash read latency and bus transfer time) and power-hungry (DRAM refresh current drawn continuously while the device is active).

The core industry challenge is therefore to shrink the "time-to-first-inference" and standby power without sacrificing model accuracy or storage capacity for larger models.

2. Application of Engineering Concepts

NAND Flash offers the highest density and lowest cost per bit and is non-volatile, but read latency (microseconds) and limited program/erase endurance make it unsuitable as a direct compute-adjacent memory, and repeated full-model reads at every power-up add measurable delay.

DRAM provides the fast, byte-addressable access needed for runtime activations and intermediate feature maps during inference, but is volatile and consumes continuous refresh power, which is costly for an always-idle, occasionally-wake edge device.

RRAM (resistive RAM) is non-volatile like flash but with access latency and endurance much closer to DRAM, no refresh power, and — critically — supports in-memory/compute-in-memory (analog MAC) operation, allowing weights to be read (or even computed upon) without first copying them into DRAM.

Technology	Latency	Volatility	Power Behavior	Role
NAND Flash	~10–100 μ s	Non-volatile	Low standby, higher active read energy	Bulk model/dataset storage
DRAM	~10–20 ns	Volatile	Continuous refresh power	Runtime activations/buffers
RRAM	~10–50 ns	Non-volatile	Near-zero standby power	Fast-load / in-memory weights

3. Solution Design

The optimized structure minimizes both wake-up latency and active power by moving frequently used model weights off flash and onto RRAM, close to the compute engine.

- Tier 1 — RRAM weight cache: store the quantized, frequently used layers of the inference model (or the entire model, if it fits) directly in RRAM, which retains data with zero standby power, eliminating the flash-to-DRAM copy step at every wake event and enabling near-instant "model-ready" state.
- Tier 2 — Small DRAM working buffer: retained only for transient activation maps and intermediate results during active inference, and power-gated (self-refresh or fully off) whenever the device is idle.

- Tier 3 — NAND Flash as the cold-storage tier: holds the full model repository, firmware, and any larger models not resident in RRAM, loaded into RRAM only on model updates rather than every boot.
- Where the accelerator supports it, exploit RRAM's compute-in-memory capability to perform matrix-vector multiplication for the first network layers directly within the memory array, cutting data movement between memory and the processing core — the single largest source of energy consumption in edge inference.

4. Use of Tools and Technologies

- NVSim / DESTINY memory-technology simulators to model latency, energy, and area trade-offs across NAND, DRAM, and RRAM configurations before hardware commitment.
- TensorFlow Lite Micro or similar edge-inference frameworks combined with an on-board power analyzer (e.g., Monsoon power monitor) to measure real wake-to-inference time and energy per inference.
- Vendor RRAM IP timing/endurance datasheets to validate retention and write-cycle assumptions against the device's expected duty cycle.

5. Analysis / Expected Outcome

By eliminating the flash-to-DRAM full-model reload at every wake cycle, time-to-first-inference can be reduced from the flash-read-dominated range down to the RRAM access range (tens of nanoseconds per weight fetch), while removing continuous DRAM refresh during idle periods substantially lowers average standby power — directly meeting the fast-load, low-power design goal.

Question 5: High-Performance Gaming Console

Problem: A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

1. Understanding of the Industry Problem

Large scene loading (open-world streaming, texture pop-in) requires the GPU and CPU to simultaneously pull large volumes of new asset data from storage into memory while the rendering pipeline continues producing frames at a fixed cadence (e.g., 16.6 ms per frame at 60 fps). If the sustained memory bandwidth required by asset streaming plus rendering exceeds what the DRAM subsystem and L3 cache can jointly deliver, the GPU stalls waiting for texture/geometry data, causing visible frame drops and stutter.

2. Application of Engineering Concepts

L3 cache (shared last-level cache, tens of MB on modern console SoCs) offers very high bandwidth and low latency but limited capacity — it can serve repeatedly accessed

textures/shaders extremely fast, but a large one-time burst of new scene data will not fit and will inevitably spill to DRAM.

DRAM (GDDR6/GDDR6X unified memory on consoles) provides much higher aggregate capacity and strong sustained bandwidth (several hundred GB/s) but with higher per-access latency than L3; during a scene-load burst, DRAM bandwidth becomes the shared, contended resource between the streaming-in asset data and the GPU's ongoing frame-render reads, and this contention — not raw DRAM speed alone — is usually the direct cause of the drop.

3. Solution Design

The enhancement strategy focuses on giving the render path guaranteed bandwidth and shrinking the volume of data that must cross the DRAM bus during a load burst.

- Memory-controller QoS/bandwidth partitioning: reserve a guaranteed bandwidth allocation for the GPU's frame-render read path, so asset-streaming DMA traffic cannot fully saturate the DRAM bus during scene loads.
- Larger/smarter L3 (or an "Infinity-Cache"-style large SRAM last-level cache) sized and tuned with streaming-aware replacement policy to retain the most recently streamed-in, still-visible textures, reducing repeat DRAM fetches within a frame.
- Direct storage-to-GPU streaming with on-the-fly decompression (bypassing a full CPU-mediated copy through DRAM twice) to cut the effective bytes that must transit the DRAM bus per loaded asset.
- Predictive/priority-based asset prefetch driven by player position and camera direction, spreading the DRAM bandwidth demand of scene loading across several preceding frames instead of concentrating it into a single burst frame.
- Texture and geometry compression formats optimized for streaming (block-compressed textures, mesh clustering) to reduce raw bytes moved per asset, directly lowering peak DRAM bandwidth demand during loads.

4. Use of Tools and Technologies

- GPU profilers such as NVIDIA Nsight Graphics, AMD uProf/Radeon GPU Profiler, or Microsoft PIX to capture per-frame memory-bandwidth utilization and identify the exact frames where DRAM contention causes a stall.
- gem5-GPU or architectural memory simulators to model the effect of L3 sizing and QoS partitioning changes before committing to silicon.
- In-engine frame-time and streaming-budget instrumentation (common in modern game engines) to correlate asset-load events with dropped frames.

5. Analysis / Expected Outcome

Reserving guaranteed GPU-path bandwidth combined with direct storage-to-GPU streaming reduces the peak DRAM bandwidth demanded during a scene load, keeping the render path's required throughput within budget even while streaming is active; spreading loads across preceding frames via predictive prefetch converts a single large burst (the direct cause of the visible drop) into a series of sub-frame-budget increments, restoring consistent frame delivery.